

Anonymous Lightweight Device-to-Device Continuous Authentication Protocol for IoT

Yaovi Ahadjitse

Université du Québec en Outaouais, Canada
ahay01@uqo.ca

Abstract—Continuous authentication is proposed as an alternative to the current methods. It is one of the core tenets of the Zero Trust Architecture model. Several continuous authentication protocols have been proposed for resource-constrained Internet of Things devices. However, the vast majority of them are designed for human/machine interaction, or use computationally expensive methods such as device fingerprinting and artificial intelligence. Others are designed for device-to-gateway communication scenarios. In this study, we present a new anonymous device-to-device continuous authentication protocol for resource-constrained Internet of Things devices. It is lightweight and uses simple cryptographic primitives such as XOR and HMAC. Our protocol generates dynamic session keys based on cubic spline polynomials and message transmission delays. Additionally, new temporary identities are generated for each continuous authentication phase. We successfully conducted an informal security analysis and an automated verification with Scyther to validate the security properties of the protocol and its robustness against known attacks. A performance comparison with recent works showed that the protocol requires less computation overhead, and its higher communication cost is justified by the values exchanged to preserve the anonymity of the devices.

Index Terms—Internet of Things, Continuous authentication, Zero-Trust Architecture, Security, HMAC.

I. INTRODUCTION

Internet of Things (IoT) technologies are enabling more and more information exchange between devices, especially through device-to-gateway [1] and device-to-device [2] communications. Authentication protocols play a key role in securing these communications. Despite numerous research on authentication protocols, the adversary can still access critical systems after the successful completion of the mutual authentication phase by an honest party. Recently, the continuous authentication method [3] has been proposed to prevent these attacks. This new method requires entities in a session to continuously authenticate each other's identity. It is one of the core tenets of the Zero-Trust Architecture (ZTA) [3] [4].

Several studies have been conducted on the design of new continuous authentication protocols for

IoT. The proposed schemes are based on either human/machine interaction such as collecting users' biometric data [5], device fingerprinting (e.g., Channel State Information) [6] [7], artificial intelligence (e.g., Convolutional Neural Networks) [8] [9]. However, these methods are computationally expensive for resource-constrained IoT devices, or are not applicable to device-to-device authentication scenarios. Additionally, some of the proposed schemes are designed for device-to-gateway authentication scenarios [1] [10]. Device fingerprinting methods are not only computationally expensive, but they are also vulnerable to interference and noise which affect their effectiveness [11]. Moreover, they are less efficient as the number of IoT devices grows in the network. While Shah et al. [7] used a device fingerprinting method (CSI) for dynamically changing session keys with the above limitations, the scheme proposed by Zhang et al. [19] used stacked tokens but relies on static keys for key negotiation, making it potentially vulnerable to adversaries [11].

Based on the previously mentioned limitations, more studies are needed. In this work, we propose an anonymous lightweight device-to-device continuous authentication protocol for IoT. Our scheme uses cubic spline polynomials, and message transmission delays in order to generate dynamic session keys.

Our Contributions

The most important contribution of this work is the anonymity. Although IoT devices can be deployed in secured environments where they are able to communicate by using their real identities, in this study and for every continuous authentication phase, a new temporary identity is created and replaces the previous one. To the best of our knowledge, this is the first time this concept is being applied during a continuous authentication phase. This feature makes it extremely difficult for the adversary to find the actual identity of the devices. Additionally, the temporary identities are created from a one way hash function and the adversary cannot recover the plain text because the hash function is pre-image and collision resistant.

We've also made other additional contributions:

- We improved the schemes proposed by Badhib et al. [1] and Chuang et al. [13] by eliminating the recovery phase. Contrary to their device-to-gateway protocol, we propose a device-to-device scheme.
- We compared our protocol to recent works and showed that it has the lowest computation cost, despite the fact that the other schemes do not have an initialization phase. Moreover, they don't generate the additional digests, as in our case, to maintain the anonymity of the devices.
- Our protocol generates dynamic keys from cubic spline polynomials which are easy to compute compared to device fingerprinting or artificial intelligence methods. By setting the matrix of cubic spline polynomials at the bootstrapping phase, we solve the interference and efficiency issues posed by device fingerprinting methods such as the CSI. Our dynamic key generation method also improves the scheme compared to Zhang et al. [12] protocol which relies on static keys.
- An informal analysis and an automated verification with the Scyther tool proved the security properties of our protocol and its robustness against known attacks.

II. RELATED WORK

A lot of studies have been conducted to propose continuous authentication protocols for IoT. A system to authenticate users in real-time was proposed by Wang et al. [7]. A convolutional neural network models the user's behavioral data from multiple smart devices. Cariello et al. [10] proposed SMARTCOPE (Smartphone Change Of Possession Evaluation), a novel framework for continuous authentication for smartphones. In their work, the user's behavioral data are collected and analyzed. However, these schemes are based on human/machine interaction and are also computationally expensive for resource-constrained IoT devices.

Methods based on device fingerprinting, e.g., Channel State Information (CSI), have been used to propose continuous authentication protocols for IoT. Yu et al. [6] and Shah et al. [7] proposed lightweight continuous authentication protocols for IoT devices that uses CSI. However, device fingerprinting methods are computationally expensive for resource-constrained IoT devices. They are also prone to noise and interference [11], and become less efficient as the number of IoT devices grows in the network.

Continuous authentication protocols with bounded session duration have also been proposed. Badhib et al. [1] proposed a device-to-gateway continuous authentication protocol for IoT that places a hard restriction on the session duration. Chuang et al. [13] also proposed a lightweight continuous authentication protocol for IoT sensing devices with bounded session time. However this scheme is vulnerable because it models the devices' energy as a linear function. Bamasag et al. [14] proposed a continuous authentication protocol for IoT based on a secret sharing scheme. The shares are only revealed during a bounded time interval. However, these schemes cannot be deployed in some scenarios. A method leveraging stacked tokens for continuous device-to-device authentication is proposed by Zhang et al. [12]. However it does not generate dynamic keys in the key negotiation process [11].

III. SYSTEM ARCHITECTURE, SECURITY AND GOALS

The mathematical background, the system model, the adversary's capabilities, the security requirements, and the design goals are presented in this section.

A. Cubic Spline interpolation

Cubic splines [15] are piece-wise polynomial functions defined on partitioned intervals. The piece-wise curves are joined at control points called *knots*. We consider a cubic spline $S(x)$ on the set of points $x_0 < x_1 < x_2 \cdots < x_{n-1}$ from a partitioned interval $[x_0, x_{n-1}]$ which satisfies the following criteria:

- A third degree polynomial p_k interpolates every sub-interval $[x_k, x_{k+1}]$ such that:

$$p_k(x) = a_k + b_k(x) + c_k(x)^2 + d_k(x)^3$$
- $S(x)$ is $\mathcal{C}^2$ continuous which ensures the smoothness of the curve.
- $S(x_i) = y_i$ for each point of the interpolated function $y = f(x)$

B. System Model

The network in our study is composed of IoT devices with limited memory capacity and computation power. They can communicate through wireless channels. The devices also have tamper-resistant mechanisms that prevent adversaries to obtain the secret security parameters through side-channel attacks [16].

C. Threat Model

An adversary $\mathcal{A}$ with the following capabilities is considered in this study:

- $\mathcal{A}$ has total control over the communication channel.

- $\mathcal{A}$ can intercept, read and modify messages.
- $\mathcal{A}$ can replay previous messages.
- $\mathcal{A}$ can learn the hashed value of the temporary identities of the devices.

D. Security Requirements

The proposed protocol is designed to guarantee the following security requirements:

- **Confidentiality:** The data exchanged between communicating parties must be protected and must not be revealed to external parties.
- **Data integrity:** The data that are exchanged must not be tampered with by an adversary.
- **Freshness:** The protocol must guarantee that the data exchanged between devices during a session contains the necessary information that indicates their freshness.
- **Resistance to replay attacks:** The protocol must detect any messages intercepted by an adversary and replayed as a new message.
- **Resistance to man-in-the-middle attacks:** The proposed protocol must prevent man-in-the-middle attacks and detect potential messages that are forged by an adversary.
- **Resistance to cloning attacks:** Messages sent from a cloned device must be detected and discarded.
- **Resistance to session-hijacking attacks:** The protocol must be robust against session-hijacking attacks.

E. Design Goals

The proposed protocol is designed with the following properties:

- **Mutual Authentication:** The devices should be able to authenticate each other's identity at the beginning of each session.
- **Continuous Authentication:** The proposed protocol should offer a continuous authentication phase.
- **Minimum overhead:** The protocol should be lightweight with minimum computation and communication overheads.
- **Anonymity:** The proposed protocol should provide an anonymity feature which is desirable in some environments. The actual identities of the devices are kept secret to prevent their potential disclosure to the adversary. The temporary identities must also be protected.
- **Forward/Backward secrecy:** The protocol must guarantee that previous keys must not encrypt

future messages, and new keys must not decrypt past messages.

IV. PROPOSED PROTOCOL

In this section, we present the details of the proposed scheme. The protocol includes different phases: the bootstrapping phase, the initialization phase, the mutual authentication phase, and the continuous authentication phase. The notations used in this work are presented in Table I.

TABLE I
NOTATIONS.

Symbol	Description
DV_i	Device identity set during bootstrapping phase
TID_i	Temporary device identity per session
μ_i	Authentication phase
$seed$	Initial seed used to generate a random number
r	Random number set at the bootstrapping phase
IK_{key}	Symmetric key at the initialization phase
V_{pol}	Matrix of cubic spline polynomials
d_i	Index of randomly selected cubic polynomial
e_i	Random value applied to d_i , $e_i \in [v_0, v_1]$
T_{mi}	Sending/receiving times of messages
$MKey$	Symmetric key for mutual authentication
$hmac$	Hash-based Message Authentication Code
$\oplus$	Exclusive-OR operator
(a, b)	Concatenates a and b
$h(.)$	Collision-resistant one-way hash function
n_i	Temporary random numbers
CK_{key}	Symmetric key for continuous authentication
δ_i	Result after applying a value e_i to a polynomial
CA_{key}	Symmetric key for continuous authentication
ϵ	Contextual information
T_{sc}	Beginning of a continuous authentication phase
ϕ	Duration of a continuous authentication phase

A. Bootstrapping Phase

This phase is performed over a secure channel or in a secure environment. During this phase, a Systems Administrator configures different security parameters on the devices. The initial seed, a random number, the duration of a continuous authentication, and other parameters are set on the devices. The System Administrator also configures the matrix of cubic spline polynomials and the real identities of the devices.

B. Initialization Phase

During the initialization phase, the two devices exchange their temporary identities and agree on a shared secret key. As an example, let's consider two devices: DV1 and DV2. Using the seed parameter, a pseudo-random number generator, and a collision resistant hash function, DV1 generates a random number n_0 , creates a temporary identity TID_1 and sends it to DV2 as follows:

$n_0 = rand()$
 $s1 = TID_1 \oplus (r \oplus \mu_0)$
 $sn_0 = n_0 \oplus r$
 $m1 = (h(TID_1), s1, sn_0)$

DV1 notes the sending time of this message as T_{m1} . After receiving $m1$, DV2 creates a response message $m2$, using its own temporary identity TID_2 , the authentication step μ_0 , the initial random number r set at the bootstrapping phase, and sends it to DV1:

$s2 = TID_2 \oplus (r \oplus \mu_0)$
 $sn_1 = sn_0 \oplus \mu_0$
 $m2 = (h(TID_2), s2, sn_1)$

DV1 will note the receiving time of $m2$ as T_{m2} and compute the delay $\epsilon = T_{m1} - T_{m2}$, and the symmetric key $IKey$. A new message $m3$ is then created and sent to DV2 which will check the validity of $IKey$. This step completes the initialization phase of the protocol.

$n_1 = rand()$
 $IKey = (TID_1 \oplus TID_2) \oplus (r \oplus \mu_0)$
 $s11 = IKey \oplus \mu_0$
 $m3 = (h(TID_1 \oplus TID_2), s11, n_1)$

C. Mutual Authentication Phase

During the mutual authentication phase, different values are computed by the devices using the authentication step μ_1 and the initial random parameter r . A device randomly selects a cubic polynomials d_i from the matrix V_{pol} . It will also choose a value $e_i \in [v_0, v_1]$ to compute δ_i such that $\delta_i = d_i \times e_i$. Note that, v_0 and v_1 are specified during the bootstrapping phase in order to limit e_i within an acceptable range. The following values are also computed by the devices: $k2i = ((\mu_1 \oplus d_i), (\mu_1 \oplus e_i))$ and $z2i = (TID_1 \oplus TID_2) \oplus (\delta_i \oplus r)$.

As an example, device DV1 generates a random number n_2 , creates a hash-based message authentication code $hmac21$, and sends a message to device DV2 as follows:

$n_2 = rand()$
 $k21 = ((\mu_1 \oplus d_1), (\mu_1 \oplus e_1))$
 $z21 = (TID_1 \oplus TID_2) \oplus (\delta_1 \oplus r)$
 $q21 = h(TID_1, TID_2)$
 $hmac21 = hmac_{IKey}(q21, k21, z21)$
 $m21 = (q21, k21, z21, hmac21, n_2)$

Device DV2 will compute $z21'$ after receiving $m21$, and successfully authenticates DV1 if $z21' \stackrel{?}{=} z21$. DV2 will then select another cubic polynomial $d_j \neq d_i$, and compute δ_2 . DV2 will also create a shared

symmetric key, $MKey$, and send a response message to DV1 as follows:

$n_3 = rand()$
 $k22 = ((\mu_1 \oplus d_2), (\mu_1 \oplus e_2))$
 $z22 = (TID_1 \oplus TID_2) \oplus (\delta_2 \oplus r)$
 $MKey = (\delta_1 \oplus \delta_2) \oplus (r \oplus \mu_1)$
 $q22 = h(TID_1, TID_2)$
 $hmac22 = hmac_{MKey}(q22, k22, z22)$
 $m22 = (q22, k22, z22, hmac22, n_3)$

After receiving $m22$, device DV1 computes $z22'$ and verifies if it matches $z22$. If these two values are equal, then DV1 successfully authenticates DV2. A new mutual authentication process is triggered otherwise.

D. Continuous Authentication Phase

A continuous authentication phase must leverage a contextual information (e.g., time, security posture, location, etc.) as the current state of the device. In this study, we used the messages' transmission delays or ϵ . Additionally, for every continuous authentication phase, using the seed and authentication step μ_2 that were set at the bootstrapping phase, a new temporary identity is created by the devices as follows: $nw_{TID_i} = h(PRNG(seed))$, $old_{TID_i} = TID_i$, $wtid_i = (nw_{TID_i} \oplus \mu_2) \oplus old_{TID_i}$, $TID_i = nw_{TID_i}$. Device DV1 will apply the transmission delay ϵ to a random cubic polynomial d_3 to compute $\delta_3 = d_3 \times \epsilon$. It will then set the time T_{sc} as the beginning of the continuous authentication phase. DV1 will send a new message $m31$ to DV2 as follows:

$n_4 = rand()$
 $k31 = ((\mu_2 \oplus d_3), (r \oplus \epsilon))$
 $z31 = (TID_1 \oplus TID_2) \oplus (\delta_3 \oplus \mu_2)$
 $q31 = h(TID_1, TID_2)$
 $hmac31 = hmac_{MKey}(q31, k31, z31, wtid_1)$
 $m31 = (q31, k31, z31, wtid_1, hmac31, n_4)$

DV1 will also save the sending time of $m31$ as T_{m31} . After receiving this message, DV2 successfully authenticates DV1 if $z31' \stackrel{?}{=} z31$. Device DV2 then computes δ_4 , the symmetric key $CKey$, and sends the following message $m32$ to DV1:

$n_5 = rand()$
 $k32 = ((\mu_2 \oplus d_4), (r \oplus \epsilon))$
 $z32 = (TID_1 \oplus TID_2) \oplus (\delta_4 \oplus \mu_2)$
 $q32 = h(TID_1, TID_2)$
 $CKey = (\delta_3 \oplus \delta_4) \oplus r$
 $hmac32 = hmac_{CKey}(q32, k32, z32, wtid_2)$
 $m32 = (q32, k32, z32, wtid_2, hmac32, n_5)$

After receiving the message if $z32' \stackrel{?}{=} z32$ then

DV1 successfully authenticate DV2. DV1 also sets the receiving time of m_{32} as $T_{m_{32}}$ and computes a new value for the transmission delay $\epsilon = T_{m_{31}} - T_{m_{32}}$. This new transmission delay will be used during the next continuous authentication phase. The two devices will start another continuous authentication phase if $(Current_time - T_{sc}) \geq \phi$.

V. SECURITY ANALYSIS

A. Informal Security Analysis

In this section, we present the informal security analysis of our proposed protocol which proves its robustness against known attacks. We also explain how the devices' identities are preserved from potential disclosure to adversaries.

- **Replay Attacks**

The proposed protocol is robust against replay attacks. To prevent this attack, new random numbers are used in our design. For example, during the mutual authentication phase, the adversary cannot replay messages m_{21} or m_{22} because they are created with new random numbers n_2 and n_3 .

- **Forward and Backward Secrecy**

The proposed protocol achieves forward and backward secrecy. The adversary cannot extract the keys from the exchanged messages $m_1, m_2, \dots, m_{32}$. She cannot access the secret parameters either due to the tamper-resistance mechanisms that protect the devices.

- **Cloning attacks**

The adversary has no control over the transmission delay ϵ . This parameter, together with the temporary identities and the dynamically changing keys, will thwart a cloning attack. For example, an attempt will result in two similar messages with different $wtid_i$ and therefore, will be detected.

- **Session Hijacking Attacks**

A session hijacking attack will be too costly to the adversary because she doesn't know the duration ϕ of a continuous authentication phase in order to renew the attacks. Even if she succeeds in getting the digest values q_{31} or q_{32} , the dynamically changing keys will prevent this attack.

- **Man-In-The-Middle Attacks**

The proposed protocol is resistant to man-in-the-middle attacks. The adversary cannot perform a side-channel attack to get the secret parameters. Additionally, she cannot forge the values z_{21} ,

z_{22} , z_{31} or z_{32} which are created with the temporary identities, and also with different δ_i .

- **Data Integrity**

An adversary cannot modify a message without it being detected and discarded. For example, during the mutual authentication or continuous authentication phase, all the messages are protected with the HMAC values $hmac_{21}$, $hmac_{22}$, $hmac_{31}$ and $hmac_{32}$. These codes are created using dynamically changing keys $MKey$ or $CKey$, the secret parameter r , and different μ_i .

- **Anonymity**

The adversary cannot get the secret parameters due to the tamper-resistant mechanisms that protect the devices. Additionally, there is no relationship between the actual identities and the temporary IDs. Even an honest but curious party cannot get the actual identity of the other party. The temporary identities are digests of random numbers which were generated using the secret *seed* and a pseudo-random number generator function. Besides, the adversary can only retrieve the digests $h(TID_1, TID_2)$ from messages m_{21} , m_{22} , m_{31} or m_{32} . However, she cannot get the temporary identities because of the one way hash function. The security of the hash functions is determined by their preimage and collision resistance [17] [18], making it impossible for the adversary to recover the original text.

B. Security Analysis with Scyther

We verified the correctness and security properties of the proposed protocol with the Scyther tool [19]. Scyther is an automatic verification tool for cryptographic protocols under the perfect cryptography assumption. Its Security Protocol Definition Language (SPDL) language allows users to model the participants in a session as roles. Scyther uses claim events to verify the security properties of the protocols. Examples of claim events are: **Alive** which specifies the aliveness of the corresponding roles during the execution of the protocol, **Secret** which proves that a given value is confidential. Fig. 1, 2 and 3, respectively show the Scyther verification's results for the three phases of the protocol. The results indicate that, after testing our protocol for a maximum number of rounds, Scyther did not find any potential attack. The proposed protocol can be successfully deployed to perform continuous authentication in device-to-device scenarios.

It is worth nothing that a heuristic security analysis, coupled with a formal analysis are usually performed

Claim	Status	Comments
ALD2DCA_Init	Ok	No attacks within bounds.
DV1		
ALD2DCA_Init.T6	Ok	No attacks within bounds.
ALD2DCA_Init.T1	Ok	No attacks within bounds.
ALD2DCA_Init.T2	Ok	No attacks within bounds.
ALD2DCA_Init.T3	Ok	No attacks within bounds.
ALD2DCA_Init.T9	Ok	No attacks within bounds.
DV2		
ALD2DCA_Init.B1	Ok	No attacks within bounds.
ALD2DCA_Init.B3	Ok	No attacks within bounds.
ALD2DCA_Init.B24	Ok	No attacks within bounds.
ALD2DCA_Init.B25	Ok	No attacks within bounds.
ALD2DCA_Init.B29	Ok	No attacks within bounds.

Fig. 1. Scyther Output - Initialization.

Claim	Status	Comments
ALD2DCA_ContAuth	Ok	No attacks within bounds.
DV1		
ALD2DCA_ContAuth.T6	Ok	No attacks within bounds.
ALD2DCA_ContAuth.T1	Ok	No attacks within bounds.
ALD2DCA_ContAuth.T2	Ok	No attacks within bounds.
ALD2DCA_ContAuth.T3	Ok	No attacks within bounds.
ALD2DCA_ContAuth.T9	Ok	No attacks within bounds.
DV2		
ALD2DCA_ContAuth.B6	Ok	No attacks within bounds.
ALD2DCA_ContAuth.B3	Ok	No attacks within bounds.
ALD2DCA_ContAuth.B4	Ok	No attacks within bounds.
ALD2DCA_ContAuth.B5	Ok	No attacks within bounds.
ALD2DCA_ContAuth.B9	Ok	No attacks within bounds.

Fig. 3. Scyther Output - Continuous Authentication.

Claim	Status	Comments
ALD2DCA_MutAuth	Ok	No attacks within bounds.
DV1		
ALD2DCA_MutAuth.T6	Ok	No attacks within bounds.
ALD2DCA_MutAuth.T1	Ok	No attacks within bounds.
ALD2DCA_MutAuth.T2	Ok	No attacks within bounds.
ALD2DCA_MutAuth.T3	Ok	No attacks within bounds.
ALD2DCA_MutAuth.T8	Ok	No attacks within bounds.
ALD2DCA_MutAuth.T9	Ok	No attacks within bounds.
DV2		
ALD2DCA_MutAuth.B6	Ok	No attacks within bounds.
ALD2DCA_MutAuth.B3	Ok	No attacks within bounds.
ALD2DCA_MutAuth.B4	Ok	No attacks within bounds.
ALD2DCA_MutAuth.B5	Ok	No attacks within bounds.
ALD2DCA_MutAuth.B8	Ok	No attacks within bounds.
ALD2DCA_MutAuth.B9	Ok	No attacks within bounds.

Fig. 2. Scyther Output - Mutual Authentication.

to prove the security of cryptographic protocols. However, several protocols which had been analyzed throughout the lenses of these two methods were later found to have various flaws [20]. In this study, we performed a rigorous informal analysis, coupled with a correctness verification and a search for potential attacks by the Scyther tool. The absence of potential attacks and our rigorous informal analysis prove the usability of our protocol in IoT environments.

VI. PERFORMANCE EVALUATION

We evaluated the performance of our protocol by comparing its communication and computation costs to existing solutions such as Badhib et al. [1], Chuang et al. [13], and Zhang et al. [12].

A. Implementation Details

The proposed protocol was implemented using Android Studio Koala 2024 [21] with the cryptographic library CRYPTO++ version 8.6.0 [22] on an HP

machine with an AMD Ryzen 3x5300U processor, 8 Gb memory, running a Windows 11 operating system.

B. Computation Cost

In our evaluation, T_h and T_{HMAC} respectively represent one hash operation and one hmac operation. The cost of the other operations, an XOR and a concatenation, are negligible and are not included in the total cost of the schemes. Table II summarizes the computation costs of the protocols. It can be observed that the total computation cost of our protocol, $7T_h + 4T_{hmac}$, is less than the costs in the schemes proposed by Badhib et al. [1], Chuang et al. [13], and Zhang et al. [12]. Consequently, our protocol has the lowest computation cost.

C. Communication Cost

The communication cost of our protocol is also evaluated and compared with the other three schemes [1] [12] [13]. The random numbers, the temporary identities, and symmetric keys are 16 bytes long. We used the SHA-1 cryptographic hash function with a 20 bytes output size. The total communication cost of our protocol is 4736 bits. Table III summarizes the communication costs of the proposed four schemes. We note that the total communication cost of our protocol is higher. This can be justified by the constraints imposed by the anonymity property of the protocol, namely the three messages exchanged during the initialization phase and the other values in the subsequent messages m_{21} , m_{22} , m_{31} , and m_{32} .

VII. DISCUSSION

Although continuous authentication has been used in scenarios involving devices and gateways, practical applications of our scheme include direct communication between IoT devices in ad hoc settings, and the

TABLE II
COMPUTATION COST.

Scheme	Init.	Mutual	Continuous
Chuang et al.	0	$19T_h + 4T_{hmac}$	$21T_h + 3T_{hmac}$
Badhib et al.	0	$23T_h + 4T_{hmac}$	$20T_h + 3T_{hmac}$
Zhang et al.	0	$7T_h + 7T_{hmac}$	$(4*j+3) T_h + (4*j+3) T_{hmac}$
A-LD2DCA	$3T_h$	$2T_h + 1T_{hmac}$	$2T_h + 1T_{hmac}$

TABLE III
COMMUNICATION COST.

Scheme	Init.	Mutual	Continuous	Cost (bit)
Chuang et al.	0	1248	1088	2296
Badhib et al.	0	1984	1536	3520
Zhang et al.	0	1280	1920	3200
A-LD2DCA	1152	1664	1920	4736

deployment of IoT devices in a distributed computing environment. The observed communication cost can be tolerated in ad hoc settings and in environments such as a company's own network infrastructure. The cost can also be accepted in cases of intermittent communication between IoT devices.

In the future we plan to reduce the communication cost of our protocol. We'll also explore the impact of jitters on the transmission delay. Finally, we plan to implement and evaluate the protocol on a constrained IoT device such as the ARM Cortex-M.

VIII. CONCLUSION

In this study, we propose a new anonymous lightweight continuous authentication protocol for device-to-device communications in IoT. The proposed scheme represents an improvement to the existing protocols which use methods that are prone to noise or interference (e.g., Chanel State Information), are computationally expensive (e.g., artificial intelligence algorithms), or are mainly designed for device-to-gateway scenarios. The protocol generates dynamically changing session keys based on cubic spline polynomials and message transmission delays. We successfully used Scyther for the automated verification of our protocol, and to validate its security properties. A comparison with existing schemes proved that our protocol has the minimum computation cost, and its higher communication cost is justified by the values exchanged in order to preserve the anonymity of the IoT devices. The proposed protocol can be deployed for continuous device-to-device authentication in IoT environments.

REFERENCES

- [1] A. Badhib, S. Alshehri, and A. Cherif, "A Robust Device-to-Device Continuous Authentication Protocol for the Internet of Things," *IEEE Access*, vol. 9, pp. 124768–124792, 2021.
- [2] S. Yang, X. Zheng, G. Liu, and X. Wang, "IBA: A secure and efficient device-to-device interaction-based authentication scheme for Internet of Things," *Computer Communications*, vol. 200, pp. 171–181, 2023.
- [3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture." NIST, 2020.
- [4] E. Gilman and D. Barth, *Zero Trust Networks: Building Secure Systems in Untrusted Networks*, 1st ed. O'Reilly Media, Inc., 2017.
- [5] D. R. Bhuva and S. Kumar, "A novel continuous authentication method using biometrics for IOT devices," *Internet of Things*, vol. 24, p. 100927, 2023.
- [6] B. Yu, C. Yang, and J. Ma, "Continuous Authentication for the Internet of Things Using Channel State Information," in 2019 IEEE Global Communications Conference (GLOBECOM), 2019, pp. 1–6.
- [7] S. W. Shah, N. F. Syed, A. Shaghghi, A. Anwar, Z. Baig, and R. Doss, "LCDA: Lightweight Continuous Device-to-Device Authentication for a Zero Trust Architecture (ZTA)," *Computers & Security*, vol. 108, p. 102351, 2021.
- [8] Y. Wang, X. Zhang, and H. Hu, "Continuous User Authentication on Multiple Smart Devices," *Information*, vol. 14, no. 5, 2023.
- [9] C. Lai, X. Zhang, H. Zhang, G. Li, Y. Yu, and D. Zheng, "GAN Augmentation-Based Continuous Authentication for Vehicular Digital Twin," in ICC 2024 - IEEE International Conference on Communications, 2024.
- [10] N. Cariello, S. Levine, G. Zhou, B. Hoplight, P. Gasti, and K. S. Balagani, "SMARTCOPE: Smartphone Change Of Possession Evaluation for continuous authentication," *Pervasive and Mobile Computing*, vol. 97, p. 101873, 2024.
- [11] J. -S. Lee, T. -H. Chen, C. -J. Chew, P. -Y. Wang and Y. -Y. Fan, "Unconsciously Continuous Authentication Protocol in Zero-Trust Architecture Based on Behavioral Biometrics," in *IEEE Transactions on Reliability*, vol. 74, no. 2, pp. 2591–2604, June 2025.
- [12] B. Zhang, S. Yang, X. Zheng, and X. Wang, "STCA: Stacked Token-based Continuous Authentication Protocol for Zero Trust IoT," in 2024 IEEE Wireless Communications and Networking Conference (WCNC), 2024, pp. 1–6.
- [13] Y.-H. Chuang, N.-W. Lo, C.-Y. Yang, and S.-W. Tang, "A Lightweight Continuous Authentication Protocol for the Internet of Things," *Sensors*, vol. 18, no. 4, 2018.
- [14] O. O. Bamasag and K. Youcef-Toumi, "Towards Continuous Authentication in Internet of Things Based on Secret Sharing Scheme," in *Proceedings of the WESS'15: Workshop on Embedded Systems Security*, in WESS'15. New York, NY, USA: Association for Computing Machinery, 2015.
- [15] C. de Boor, *A practical guide to splines*. New York: Springer, 2001.
- [16] D. Wang, D. He, P. Wang, and C.-H. Chu, "Anonymous Two-Factor Authentication in Distributed Systems: Certain Goals Are Beyond Attainment," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 4, pp. 428–442, 2015.
- [17] A. Bogdanov, M. Knezevic, G. Leander, D. Toz, K. Varici, and I. Verbauwhede, "SPONGENT: The Design Space of Lightweight Cryptographic Hashing," *IEEE Transactions on Computers*, vol. 62, no. 10, pp. 2041–2053, 2013.
- [18] A. J. Menezes, S. A. Vanstone, and P. C. V. Oorschot, *Handbook of Applied Cryptography*, 1st ed. USA: CRC Press, Inc., 1996.
- [19] C. J. Cremers, "The Scyther Tool: Verification, Falsification, and Analysis of Security Protocols," in *Proceedings of the 20th International Conference on Computer Aided Verification*, in CAV '08. Berlin, Heidelberg: Springer-Verlag, 2008, pp. 414–418.

- [20] Q. Wang and D. Wang, "Understanding Failures in Security Proofs of Multi-Factor Authentication for Mobile Devices," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 597–612, 2023.
- [21] Google, "Android Studio and SDK Tools." Accessed: Sep. 10, 2024. [Online]. Available: <https://developer.android.com/studio>.
- [22] Weidai11, "Cryptopp Releases." Accessed: Nov. 12, 2024. [Online]. Available: <https://github.com/weidai11/cryptopp/releases>.